

Министерство науки и высшего образования Российской Федерации
Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

ОТЧЕТ

по Лабораторной работе N 6

«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся: Петрова Мария Валерьевна

Факультет: прикладной информатики

Группа: K3239

Направление подготовки: 09.03.03 Прикладная информатика

Образовательная программа: Мобильные и сетевые технологии 2023

Преподаватели: Говорова Марина Михайловна

Белов Александр

Санкт-Петербург

2025 г.

Содержание

1	Наименование работы	2
2	Цель работы	2
3	Практическое задание	2
4	Выполнение работы	2
4.1	Практическое задание 2.1.1	2
4.2	Практическое задание 2.2.1	4
4.3	Практическое задание 2.2.2	8
4.4	Практическое задание 2.2.3	9
4.5	Практическое задание 2.1.4	10
4.6	Практическое задание 2.3.1	11
4.7	Практическое задание 2.3.2	12
4.8	Практическое задание 2.3.3	12
4.9	Практическое задание 2.3.4	12
4.10	Практическое задание 3.1.1	13
4.11	Практическое задание 3.1.2	15
4.12	Практическое задание 3.2.1	16
4.13	Практическое задание 3.2.2	17
4.14	Практическое задание 3.2.3	17
4.15	Практическое задание 3.3.1	17
4.16	Практическое задание 3.3.2	18

1 Наименование работы

Работа с БД в СУБД MongoDB.

2 Цель работы

Цель лабораторной работы: овладеть практическими навыками работы с CRUD-операциями, вложенными объектами в коллекции базы данных MongoDB, агрегацией и изменением данных, а также ссылками и индексами в базе данных MongoDB.

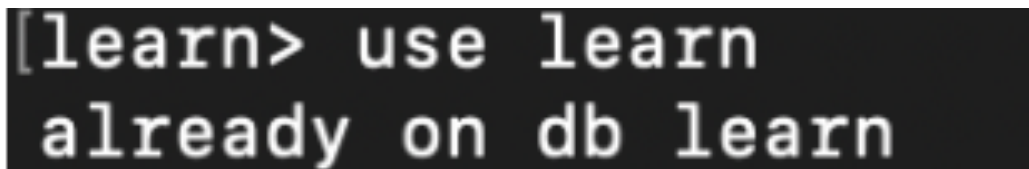
3 Практическое задание

В ходе лабораторной работы необходимо создать базу данных `learn`, заполнить коллекции `unicorns` и `towns`, выполнить запросы на выборку, сортировку, ограничение результата, работу с массивами, вложенными объектами, агрегациями и редактированием документов.

4 Выполнение работы

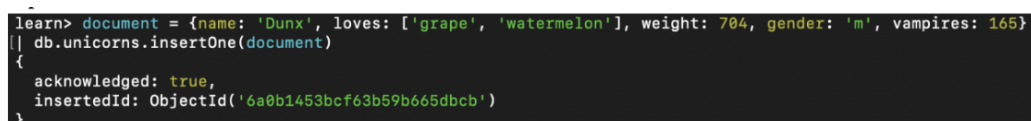
4.1 Практическое задание 2.1.1

Была создана база данных `learn`. Затем была заполнена коллекция `unicorns`. Также документ для единорога `Dunx` был добавлен вторым способом: сначала документ был сохранен в переменную, после чего вставлен в коллекцию.



```
[learn> use learn  
already on db learn
```

Рисунок 1 – Создание базы данных `learn` и начало заполнения коллекции `unicorns`



```
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}  
[| db.unicorns.insertOne(document)  
{  
  acknowledged: true,  
  insertedId: ObjectId('6a0b1453bcf63b59b665dbcb')  
}
```

Рисунок 2 – Добавление документа `Dunx` вторым способом

```

[learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc0'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc2'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc3'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc4'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {

```

Рисунок 3 – Проверка содержимого коллекции unicorns

4.2 Практическое задание 2.2.1

Были сформированы запросы для вывода списков самцов и самок единорогов. Списки были отсортированы по имени. Также список самок был ограничен первыми тремя особями.

```
[learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0b1453bcf63b59b665dbcb'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc0'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc6'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc9'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
]
```

Рисунок 4 – Список самцов, отсортированный по имени

```

[learn> db.unicorns.find({gender: 'f'}).sort({name: 1})
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc5'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc8'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbca'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc4'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]

```

Рисунок 5 – Список самок, отсортированный по имени

```

[learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc5'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc8'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]

```

Рисунок 6 – Первые три самки, отсортированные по имени

Далее были найдены самки, которые любят carrot. Для ограничения результата использовались функции findOne() и limit().

```

[learn> db.unicorns.find({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc4'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbca'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]

```

Рисунок 7 – Все самки, которые любят carrot

```

[learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}

```

Рисунок 8 – Первая самка, которая любит carrot, через findOne

```
[learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рисунок 9 – Первая самка, которая любит carrot, через limit

4.3 Практическое задание 2.2.2

Был модифицирован запрос для вывода списка самцов единорогов. Из результата были исключены поля loves и gender.

```

learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc0'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc2'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc3'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc6'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc7'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc9'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a0b1453bcf63b59b665dbcb'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]

```

Рисунок 10 – Список самцов без полей loves и gender

4.4 Практическое задание 2.2.3

Был выведен список единорогов в обратном порядке добавления с помощью сортировки по \$natural.

```

]
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a0b1453bcf63b59b665dbcb'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbca'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc9'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc8'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
]

```

Рисунок 11 – Список единорогов в обратном порядке добавления

4.5 Практическое задание 2.1.4

Был выведен список единорогов с названием первого любимого предпочтения. Для этого использовался оператор `$slice`. Идентификатор `_id` был исключен из результата.

```
learn> db.unicorns.find({}, {_id: 0, name: 1, loves: {$slice: 1}})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
```

Рисунок 12 – Список единорогов с первым любимым предпочтением

4.6 Практическое задание 2.3.1

Был выведен список самок единорогов весом от 500 до 700 кг. Поле `_id` было исключено из вывода.

```
learn> db.unicorns.find(
|   {gender: 'f', weight: {$gte: 500, $lte: 700}},
|   {_id: 0}
| )
[
|   {
|     name: 'Solnara',
|     loves: [ 'apple', 'carrot', 'chocolate' ],
|     weight: 550,
|     gender: 'f',
|     vampires: 80
|   },
|   {
|     name: 'Leia',
|     loves: [ 'apple', 'watermelon' ],
|     weight: 601,
|     gender: 'f',
|     vampires: 33
|   },
|   {
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
```

Рисунок 13 – Самки единорогов весом от 500 до 700 кг

4.7 Практическое задание 2.3.2

Был выведен список самцов единорогов весом от 500 кг, которые одновременно предпочитают grape и lemon.

```
learn> db.unicorns.find(
|   {gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}},
|   {_id: 0}
| )
[
|   {
|     name: 'Kenny',
|     loves: [ 'grape', 'lemon' ],
|     weight: 690,
|     gender: 'm',
|     vampires: 39
|   }
| ]
```

Рисунок 14 – Самцы весом от 500 кг, предпочитающие grape и lemon

4.8 Практическое задание 2.3.3

Были найдены единороги, у которых отсутствует ключ vampires. Для этого использовался оператор \$exists.

```
learn> db.unicorns.find(
|   {vampires: {$exists: false}}
| )
[
|   {
|     _id: ObjectId('6a0b141ebcf63b59b665dbca'),
|     name: 'Nimue',
|     loves: [ 'grape', 'carrot' ],
|     weight: 540,
|     gender: 'f'
|   }
| ]
```

Рисунок 15 – Единороги без ключа vampires

4.9 Практическое задание 2.3.4

Был выведен упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

```
learn> db.unicorns.find(
|   {gender: 'm'},
|   {_id: 0, name: 1, loves: {$slice: 1}}
| ).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

Рисунок 16 – Упорядоченный список самцов с первым предпочтением

4.10 Практическое задание 3.1.1

Была создана коллекция `towns`, содержащая документы с информацией о городах, населении, известных особенностях и мэрах.

```

learn> db.towns.insertMany([
|   {
|     name: "Punxsutawney",
|     population: 6200,
|     last_sensus: ISODate("2008-01-31"),
|     famous_for: [""],
|     mayor: {
|       name: "Jim Wehrle"
|     }
|   },
|   {
|     name: "New York",
|     population: 22200000,
|     last_sensus: ISODate("2009-07-31"),
|     famous_for: ["status of liberty", "food"],
|     mayor: {
|       name: "Michael Bloomberg",
|       party: "I"
|     }
|   },
|   {
|     name: "Portland",
|     population: 528000,
|     last_sensus: ISODate("2009-07-20"),
|     famous_for: ["beer", "food"],
|     mayor: {
|       name: "Sam Adams",
|       party: "D"
|     }
|   }
| ])
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6a0b196dbcf63b59b665dbcc'),
|     '1': ObjectId('6a0b196dbcf63b59b665dbcd'),
|     '2': ObjectId('6a0b196dbcf63b59b665dbce')
|   }
| }
| }

```

Рисунок 17 – Создание коллекции towns

С помощью обращения к вложенному полю mayor.party был сформирован запрос, который возвращает города с независимыми мэрами, где party = "I".

```
learn> db.towns.find(
|   {"mayor.party": "I"},
|   {_id: 0, name: 1, mayor: 1}
| )
|
| { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
|
```

Рисунок 18 – Города с независимыми мэрами

Также был сформирован запрос, который возвращает города с беспартийными мэрами, то есть документы, где поле `mayor.party` отсутствует.

```
learn> db.towns.find(
|   {"mayor.party": {$exists: false}},
|   {_id: 0, name: 1, mayor: 1}
| )
|
| { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

Рисунок 19 – Города с беспартийными мэрами

4.11 Практическое задание 3.1.2

Была сформирована функция для вывода списка самцов единорогов.

```
learn> fn = function() { return this.gender == "m"; }
[Function: fn]
```

Рисунок 20 – Создание функции для отбора самцов

Так как в текущей версии `mongosh` передача функции напрямую в `find()` не поддерживается, курсор был создан с использованием обычного фильтра `{gender: "m"}`. Курсор содержит первых двух самцов, отсортированных в лексикографическом порядке.

```
learn> var cursor = db.unicorns.find({gender: "m"}).sort({name: 1}).limit(2); null;
null
```

Рисунок 21 – Создание курсора для первых двух самцов

Результат был выведен с помощью метода `forEach`.

```
learn> cursor.forEach(function(obj) {
|   print(obj.name);
[| })
Dunx
Horny
```

Рисунок 22 – Вывод результата через forEach

Также было проверено содержание коллекции unicorns.

```
[learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc0'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
]
```

Рисунок 23 – Содержание коллекции unicorns

4.12 Практическое задание 3.2.1

Было найдено количество самок единорогов весом от 500 до 600 кг.

```
learn> db.unicorns.countDocuments({
|   gender: 'f',
|   weight: {$gte: 500, $lte: 600}
[| })
2
```

Рисунок 24 – Количество самок весом от 500 до 600 кг

4.13 Практическое задание 3.2.2

С помощью метода `distinct()` был выведен список уникальных предпочтений единорогов.

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Рисунок 25 – Список предпочтений единорогов

4.14 Практическое задание 3.2.3

С помощью агрегированного запроса было посчитано количество единорогов каждого пола.

```
learn> db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: {$sum: 1}
    }
  }
])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
```

Рисунок 26 – Количество единорогов обоих полов

4.15 Практическое задание 3.3.1

Был добавлен новый единорог Barny. В методичке указан метод `save()`, однако в текущей версии `mongosh` он не поддерживается, поэтому был использован современный метод `insertOne()`.

```
learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a0b1c28bcf63b59b665dbcf')
}
```

Рисунок 27 – Добавление единорога Barney

После добавления было проверено содержимое коллекции.

```
learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a0b1c28bcf63b59b665dbcf'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

Рисунок 28 – Проверка добавления Barney

4.16 Практическое задание 3.3.2

Для самки единорога Айна были изменены значения полей: вес стал равен 800, а количество убитых вампиров стало равно 51.

```
learn> db.unicorns.updateOne(
|   {name: 'Ayna'},
|   {$set: {weight: 800, vampires: 51}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рисунок 29 – Изменение данных Айна

```
}  
[learn> db.unicorns.find({name: 'Ayna'})  
[  
  {  
    _id: ObjectId('6a0b141ebcf63b59b665dbc5'),  
    name: 'Ayna',  
    loves: [ 'strawberry', 'lemon' ],  
    weight: 800,  
    gender: 'f',  
    vampires: 51  
  }  
]
```

Рисунок 30 – Проверка данных Ауна

4.17 Практическое задание 3.3.3

Для самца единорога Raleigh было изменено поле loves: теперь он любит redbull.

```
learn> db.unicorns.updateOne(
|   {name: 'Raleigh'},
|   {$set: {loves: ['redbull']}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рисунок 31 – Изменение предпочтений Raleigh

После изменения данных было проверено содержимое коллекции unicorns.

```
[learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc7'),
    name: 'Raleigh',
    loves: [ 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Рисунок 32 – Проверка данных Raleigh

4.18 Практическое задание 3.3.4

Всем самцам единорогов было увеличено количество убитых вампиров на 5 с помощью оператора \$inc.

```
learn> db.unicorns.updateMany(  
  |   {gender: 'm'},  
  |   {$inc: {vampires: 5}}  
[ | )  
  {  
    acknowledged: true,  
    insertedId: null,  
    matchedCount: 8,  
    modifiedCount: 8,  
    upsertedCount: 0  
  }  
]
```

Рисунок 33 – Увеличение количества vampires у самцов

После выполнения обновления было проверено содержимое коллекции unicorns.

```
learn> db.unicorns.find({gender: 'm'})
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc0'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc2'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc3'),
    name: 'Roooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 104
  },
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc6'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 44
  },
]
```

Рисунок 34 – Проверка обновленных данных самцов

4.19 Практическое задание 3.3.5

Была изменена информация о городе Портланд: мэр этого города стал беспартийным. Для этого было удалено поле `mayor.party` с помощью оператора `$unset`.

```

learn> db.towns.updateOne(
|   {name: 'Portland'},
|   {$unset: {'mayor.party': 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>

```

Рисунок 35 – Удаление поля party у мэра Portland

После изменения данных было проверено содержимое коллекции towns.

```

learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a0b196dbcf63b59b665dbce'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]

```

Рисунок 36 – Проверка данных Portland

4.20 Практическое задание 3.3.6

Для самца единорога Pilot в массив предпочтений было добавлено значение chocolate.

```
learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рисунок 37 – Добавление chocolate в предпочтения Pilot

После изменения данных было проверено содержимое коллекции unicorns.

```
[learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc9'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Рисунок 38 – Проверка данных Pilot

4.21 Практическое задание 3.3.7

Для самки единорога Aurora в массив предпочтений были добавлены значения sugar и lemon с помощью оператора \$addToSet.

```
learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Рисунок 39 – Добавление sugar и lemon в предпочтения Aurora

После изменения данных было проверено содержимое коллекции unicorns.

```
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a0b141ebcf63b59b665dbc1'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Рисунок 40 – Проверка данных Aurora

4.22 Практическое задание 3.4.1

Была создана коллекция towns для выполнения операций удаления.

```

learn> db.towns.insertMany([
  {
    name: "Punxsutawney",
    population: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: {
      name: "Jim Wehrle"
    }
  },
  {
    name: "New York",
    population: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: {
      name: "Michael Bloomberg",
      party: "I"
    }
  },
  {
    name: "Portland",
    population: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: {
      name: "Sam Adams",
      party: "D"
    }
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6a0b1e15bcf63b59b665dbd0'),
    '1': ObjectId('6a0b1e15bcf63b59b665dbd1'),
    '2': ObjectId('6a0b1e15bcf63b59b665dbd2')
  }
}

```

Рисунок 41 – Создание коллекции towns для удаления данных

Затем были удалены документы с беспартийными мэрами.

```

learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 1 }

```

Рисунок 42 – Удаление документов с беспартийными мэрами

После удаления было проверено содержание коллекции.

```
[learn> db.towns.find()
[
  {
    _id: ObjectId('6a0b1e15bcf63b59b665dbd1'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a0b1e15bcf63b59b665dbd2'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

Рисунок 43 – Проверка коллекции towns после удаления

Далее коллекция была очищена.

```
[learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
```

Рисунок 44 – Очистка коллекции towns

После этого был просмотрен список доступных коллекций.

```
[learn> show collections
towns
unicorns
```

Рисунок 45 – Список доступных коллекций

4.23 Практическое задание 4.1.1

Была создана коллекция зон обитания единорогов. В качестве идентификатора использовалось краткое название зоны, также были добавлены полное

название и описание.

```
learn> db.habitats.insertMany([
  {
    _id: "forest",
    name: "Magic Forest",
    description: "Лесная зона обитания единорогов"
  },
  {
    _id: "mountain",
    name: "Crystal Mountains",
    description: "Горная зона обитания единорогов"
  },
  {
    _id: "lake",
    name: "Silver Lake",
    description: "Озёрная зона обитания единорогов"
  }
])
{
  acknowledged: true,
  insertedIds: { '0': 'forest', '1': 'mountain', '2': 'lake' }
}
```

Рисунок 46 – Создание коллекции зон обитания

Для нескольких единорогов были добавлены ссылки на зоны обитания с использованием второго способа автоматического связывания.

```

learn> db.unicorns.updateOne(
|   {name: "Aurora"},
|   {$set: {habitat: {$ref: "habitats", $id: "forest"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.updateOne(
|   {name: "Unicrom"},
|   {$set: {habitat: {$ref: "habitats", $id: "mountain"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.updateOne(
|   {name: "Nimue"},
|   {$set: {habitat: {$ref: "habitats", $id: "lake"}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }

```

Рисунок 47 – Добавление ссылок на зоны обитания

Затем было проверено содержание коллекции unicorns.

```

learn> db.unicorns.find(
|   {habitat: {$exists: true}},
|   {_id: 0, name: 1, habitat: 1}
| )
|
| [
|   { name: 'Aurora', habitat: DBRef('habitats', 'forest') },
|   { name: 'Unicrom', habitat: DBRef('habitats', 'mountain') },
|   { name: 'Nimue', habitat: DBRef('habitats', 'lake') }
| ]

```

Рисунок 48 – Проверка ссылок на зоны обитания

4.24 Практическое задание 4.2.1

Была проверена возможность создания уникального индекса для поля name в коллекции unicorns.

```
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
```

Рисунок 49 – Создание уникального индекса по полю name

После создания индекса была выполнена проверка ограничения уникальности. При попытке добавить документ с уже существующим именем MongoDB выдала ошибку дублирования ключа. Это означает, что уникальный индекс работает корректно.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Рисунок 50 – Проверка уникального индекса

4.25 Практическое задание 4.3.1

Была получена информация обо всех индексах коллекции unicorns.

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

Рисунок 51 – Информация об индексах коллекции unicorns

После этого были удалены все пользовательские индексы, кроме индекса идентификатора.

```
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
```

Рисунок 52 – Удаление пользовательских индексов

Также была выполнена попытка удалить индекс `_id`. MongoDB не позволяет удалить этот индекс, так как он является системным и создается автоматически.

```
learn> db.unicorns.dropIndex("_id")
MongoServerError[InvalidOptions]: cannot drop _id index
learn>
```

Рисунок 53 – Попытка удаления индекса `_id`

4.26 Практическое задание 4.4.1

Была создана объемная коллекция `numbers`, содержащая 100000 документов.

```
learn> for (let i = 0; i < 100000; i++) {
|   db.numbers.insertOne({value: i})
| }
| {
|   acknowledged: true,
|   insertedId: ObjectId('6a0b217bbcf63b59b6676273')
| }
```

Рисунок 54 – Создание коллекции numbers

После этого были выбраны последние четыре документа.

```
[learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a0b217bbc63b59b6676273'), value: 99999 },
  { _id: ObjectId('6a0b217bbc63b59b6676272'), value: 99998 },
  { _id: ObjectId('6a0b217bbc63b59b6676271'), value: 99997 },
  { _id: ObjectId('6a0b217bbc63b59b6676270'), value: 99996 }
]
```

Рисунок 55 – Выбор последних четырех документов

Для анализа плана выполнения запроса был использован метод `explain("executionStats")`.

```
learn> db.numbers.explain("executionStats").find().sort({value: -1}).limit(4)
{
  "sumOfExecTimeMillis": 111
```

Рисунок 56 – План выполнения запроса без индекса

В плане выполнения запроса без индекса значение `executionTimeMillis` составило 53 мс. Также было видно, что MongoDB просмотрела 100000 документов.

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 53,
  totalKeysExamined: 0,
  totalDocsExamined: 100000,
  executionStages: {
    isCached: false,
    stage: 'SORT',
    nReturned: 4,
    executionTimeMillisEstimate: 44,
    works: 100006,
    advanced: 4,
    needTime: 100001,
    needYield: 0,
  }
}

```

Рисунок 57 – Значение executionTimeMillis без индекса

Затем был создан индекс по полю value.

```

[learn> db.numbers.createIndex({value: 1})
value_1

```

Рисунок 58 – Создание индекса по полю value

После создания индекса была получена информация обо всех индексах коллекции numbers.

```

[learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]

```

Рисунок 59 – Индексы коллекции numbers

Затем запрос на выбор последних четырех документов был выполнен повторно.

```

learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a0b217bbcf63b59b6676273'), value: 99999 },
  { _id: ObjectId('6a0b217bbcf63b59b6676272'), value: 99998 },
  { _id: ObjectId('6a0b217bbcf63b59b6676271'), value: 99997 },
  { _id: ObjectId('6a0b217bbcf63b59b6676270'), value: 99996 }
]

```

Рисунок 60 – Выбор последних четырех документов после создания индекса

После этого был снова проанализирован план выполнения запроса с установленным индексом.

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 0,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 4,
    executionTimeMillisEstimate: 0,
    works: 5,
    advanced: 4,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    limitAmount: 4,
    inputStage: {

```

Рисунок 61 – План выполнения запроса с индексом

После создания индекса время выполнения запроса уменьшилось с 53 мс до 0 мс. Количество просмотренных документов уменьшилось со 100000 до 4, а количество просмотренных ключей индекса составило 4. Следовательно, запрос

с использованием индекса является более эффективным.

5 Выводы

В ходе второй части лабораторной работы были выполнены операции редактирования, удаления документов, создания ссылок и настройки индексов в MongoDB. Были изменены данные в коллекциях `unicorns` и `towns`, выполнено удаление документов и очистка коллекции.

Также была создана коллекция зон обитания `habitats`, после чего для нескольких единорогов были добавлены ссылки на соответствующие зоны. В работе с индексами был создан уникальный индекс по полю `name`, проверена ошибка дублирования ключа, а также изучено удаление пользовательских индексов и невозможность удаления системного индекса `_id`.

На примере коллекции `numbers` было показано, что индекс по полю `value` значительно ускоряет выполнение запроса: время выполнения уменьшилось с 53 мс до 0 мс, а количество просмотренных документов уменьшилось со 100000 до 4.